Precise Point Positioning

1. Minimum System Requirements

- Processor: Multi-core processor (e.g., Intel Core i3 or AMD Ryzen 3)
- RAM: 4 GB
- Storage: Standard hard disk drive (HDD) or solid-state drive (SSD)
- Operating System: Linux distribution (e.g., Ubuntu, Fedora, CentOS)

2. How to install Python?

- Download Miniconda installer from: https://docs.conda.io/en/latest/miniconda.html#linux-installers
- In your terminal, run: `bash Miniconda3-latest-Linux-x86_64.sh`
- You can refer to https://docs.conda.io/projects/conda/en/latest/user-guide/install/linux.html for a detailed Miniconda installation guide.
- After Miniconda has been installed, install **python version 3.9.7** by running:

  ```
  conda create --name py3 python=3.9.7
  ```

  Note that it is important to download this specific version of python, so as to make all the required library versions compatible with each other.
- After python installation is done, you can run `conda activate py3` to verify the environment.

3. How to install required libraries?

- Navigate to the file location where this README is placed and open command prompt.
- Activate the python environment by running: `conda activate py3.` This will allow the libraries to be installed for this specific Python environment.
- Install python packages using: `pip install -r requirements.txt.` The `requirements.txt` file is placed in the same file location where this readme is placed
- After all the python packages has been installed successfully, you can list them using: `pip list`

4. How to run the POD program?

- Navigate to the file location where this README is placed and open command prompt. Note that running the code from any other location will result in improper functioning.
- Run `python POD/POD.py -h` for help. This will show you the input parameter arguments required along with the argument tags. The usage and arguments are listed as follows:

```
usage: POD.py [-h] [-r RINEX] [-k KINEMATIC] [-m METHOD] [-s SATELLITE] [-i
ITERATIONS] [-j J1J2] [-f SMOOTHENING]
              [-o OUTPUT]

**Precise point Positioning**

optional arguments:
  -h, --help            Welcome to our code
  -r RINEX, --RINEX RINEX
                        RINEX input file location
  -k KINEMATIC, --kinematic KINEMATIC
                        Kinematic File location. If nothing is provided, only
estimated state will be saved
  -m METHOD, --method METHOD
                        code for using Code measurements ,carrier for using
             Carrier measurements
  -s SATELLITE, --satellite SATELLITE
                        Satellite Name (optional)
  -i ITERATIONS, --iterations ITERATIONS
                        Number of Iterations ,Default = Full
  -j J1J2, --J1J2 J1J2  Set 1 to calculate J1,J2 ,Default = False
  -f SMOOTHENING, --smoothening SMOOTHENING
                        Set 0 to disable smoothing of output ,Default = True
  -o OUTPUT, --output OUTPUT
                        Explicitly define the output path ,Default =
'POD/output_files/'
```

The arguments are further detailed:

- `-r`: You need to provide the RINEX file location here.
- `-k`: You need to provide the kinematic orbit file location here. It is an optional tag, and if nothing is provided, only position estimate from EKF will be stored in output
- `-m`: It denotes the method of the algorithm that will run for position estimation. You can specify `code` for `CodeFinetune` algorithm and `carrier` for `CarrierFinetune` algorithm. If nothing is mentioned, `CarrierFinetune` algorithm will be automatically selected.

- `-s`: Satellite name for which precise orbit determination is to be performed.
- `-i`: Number of iterations, for which the algorithm will run. Default iteration is selected as `Full`.
- `-j`: If this argument is set to 1, the sensitivity and robustness metrics will be calculated. Default for this argument is `False`.
- `-f`: If this argument is set to 0, smoothing of estimated position will not happen. Default for this argument is `True`.
- `-o`: It specifies the output location where the results will be stored. If nothing is specified it will store to default location: `POD/output_files/.` Note that inside this output folder, another folder with the date mentioned in RINEX will be created automatically, and within that folder another one will be created with method name and date-time, when the program was run. All the results will be stored in this particular folder. Sample output location: `POD\output_files\2016-11-30\CarrierFinetune_20230501120551` The different sample files are listed below, which are stored:
    - Folder J1J2_testing: Contains _J1.csv, _J2.csv, _J1k.csv, _J2k.csv. These contain the J1 and J2 metrics values. If J1, J2 flag is disabled, this will not be generated.
    - CarrierFinetune_Residue.csv **[1]**: This file contains the measurement residues calculated at the a priori stage of the filter.
    - CarrierFinetune_Residue_post.csv **[1]**: This file contains the measurement residues calculated at the a posteriori stage of the filter.
    - CarrierFinetune_residue_post_EKF.pdf **[1]**: This file contains the measurement residue plot which is calculated at the a posteriori stage of the filter.
    - CarrierFinetune_Residue_Stat.csv **[1]**: This file contains the mean and standard deviation for residues of each satellite calculated at the a priori stage of the filter.
    - CarrierFinetune_Residue_Stat_post.csv **[1]**: This file contains the mean and standard deviation for residues of each satellite calculated at the a posteriori stage of the filter.
    - Correct_Observations.pdf: This file contains the plot for number of correct observations throughout the entire time span for which the algorithm was run.
    - Fault_Observation.csv: Contains the faulty observation timestamp, with other details.
    - Grace_Correct_state_ecef.csv **[2]**: Contains the smoothened position estimate of the receiver satellite. The fixed interval smoothing algorithm is used to

smoothen the Extended Kalman Filter (EKF) position estimate. If smoothing is disabled, the unsmoothed state will be stored in this file.

- Grace_Correct_state_ecef_unsmoothed.csv [2]: Contains the unsmoothed position estimate of the receiver satellite derived from EKF. If smoothing is disabled, this file will not be generated.
- Grace_Position_Error_EKF_CarrierFinetune.pdf [1] [2] [3]: This file contains the position (x, y, z) error plots in ECEF frame along with the norm error. The norm error is determined by taking the square root of the sum of the squared errors in the x, y, and z dimensions.
- Grace_Position_Error_with_Range_EKF_CarrierFinetune.pdf [1] [2] [3]: This file contains the position (x, y, z) error plots in ECEF along with the range error. Range error is the error which is calculated by subtracting norm of true position vector and norm of estimated position vector.
- hist_range.png [3]: Histogram of range error.
- hist_x.png [3]: Histogram of error along x-axis.
- hist_y.png [3]: Histogram of error along y-axis.
- hist_z.png [3]: Histogram of error along z-axis.
- metadata.txt: This file contains some of the relevant information and parameters regarding the algorithm like method, iterations, date etc.
- Pos_error.csv [3]: Stores the x, y, z position error values along with standard deviation (STD) values derived from EKF.
- Pos_Range_stat.csv [3]: Stores the statistics (mean, STD, etc) for position errors and range error.
- Range_error.csv [3]: Stores the range error values.
- Range_error_for_CarrierFinetune.png [1] [3]: This file contains the plot for range error.

[1]: The file name contains ‘CarrierFinetune’ because ‘carrier’ was selected as method. If ‘code’ was selected, it would have been ‘CodeFinetune’.

[2]: The file contains ‘Grace’ because satellite name was given as ‘Grace’.

[3]: This file will not be generated if kinematic file is not provided.

Some sample runs:

- `python POD/POD.py -r "RNX_file_path" -k "Kinematic_file_path" -m "code" -s "Grace" :` This will run the algorithm using code measurements for Grace and provide all the error plots as kinematic file path is also provided. This will run for full time span, as nothing is specified and the output storage location will also be default.
- `python POD/POD.py -r "RNX_file_path" -m "code" -s "Grace" :` This will run the algorithm using code measurements for Grace but will not provide any error plots. This will also run for full time span.
- `python POD/POD.py -r "RNX_file_path" -k "Kinematic_file_path" -m "carrier" -s "Grace" -i 1000 :` This will run the algorithm using carrier measurements for Grace and provide all the error plots as kinematic file path is also provided. This will run for 1000 iterations.
- `python POD/POD.py -r "RNX_file_path"  -m "carrier" -s "Grace" -f 0 :` This will run the algorithm using carrier measurements for Grace but will not provide any error plots. This will run for full time span and no smoothing of estimate will happen.

Note that the code will only run if the terminal is open in the file location, where this README is placed.

In case any kind of plot generation needs to be done manually at post processing, i.e., after you have the Satellite position estimate, the `misc.py` module can be used to do so. For help regarding use of `misc.py`, run `python POD/modules/misc.py –h` in command prompt from the file location where this readme is placed.

5. Code Structure

- **POD_Respond:** The root folder where the instructions.txt, requirements.txt and README files are placed.
  - **POD:** Contains the main POD.py file.
    - **input_files:** Contains the RINEX and kinematic orbit file along with GPS input files required for the algorithm to run. Whenever .sp3 or GPS broadcast ephemeris files are downloaded, they are automatically stored here.
    - **output_files:** Default location for all output results.
    - **modules:** Contains all the relevant python codes that are used in the algorithm.

6. Code Configuration

All the required configurations are done in the `constants.py file`. If you need to change a particular configuration, you can do so by editing the `constants.py` file, opening it from any text editor such as Notepad or Notepad++. Some of the configurations are listed:

- `self.th_init:` The initial measurement residue threshold value beyond which that particular measurement is filtered out. Applicable for iterations less than 50.
- `self.th_transition:` The transitioning measurement residue threshold value beyond which that particular measurement is filtered out. Applicable from 50th to 749th iteration.
- `self.th_stable:` The stable measurement residue threshold value beyond which that particular measurement is filtered out. Applicable from 750th iteration onwards.
- `self.SNR_cutoff:` The SNR threshold value beyond which the measurement is filtered out.
- `self.model_order:` The orbit model order for state prediction.
- `self.Q_order, self.R_order:` The order to which 10 is raised and multiplied with Q or R value to get the final Q or R.
- `self.R_code, self.R_carrier:` The measurement noise covariance value for code and carrier.
- `self.P:` The initial error covariance matrix.
- `self.w:` The process noise parameters.
- `self.antexFile:` Antenna Phase Correction file location.
- `self.gravity_file:` Gravity constants file location.
- `self.orbit_file_type:` GPS orbit file type ((P)recise, (R)apid, (U)ltra-Rapid)

7. Code Modules

You can find the code description for each module in docs folder by running `index.html`.